Backend Engineering Assignment: Core API & Guardrails

- Overview : . This project is a Spring Boot microservice that acts as a central API gateway and guardrail system. It ensures safe and controlled interactions between users and bots using Redis for atomic operations and PostgreSQL as the source of truth.
- Tech Stack : Java 17+ Spring Boot 3.x PostgreSQL Redis (Spring Data Redis) Docker
- Architecture 🔹 PostgreSQL (Source of Truth) Stores : Users Bots Posts Comments 🔹 Redis (Guardrail Engine) Handles : Virality score (real-time)
- Bot interaction limits Cooldowns Notification batching 1
- Features Phase 1: Core API Endpoints • POST /api/posts → Create post POST /api/posts/{postId}/comments → Add comment POST /api/posts/{postId}/like → Like post
- Phase 2: Redis Virality Engine & Atomic Locks
- Virality Score Stored in Redis key: post:{id}:virality Action Score Bot Reply +1 Human Like +20 Human Comment +50
- Atomic Locks (Thread Safety) 1. Horizontal Cap • • •
- Max 100 bot replies per post Redis Key: post:{id}:bot_count Uses atomic 2. Vertical Cap • INCR Max depth level = 20 3. Cooldown Cap • A bot cannot interact with same user within 10 minutes 2 Redis Key: • • cooldown:{botId}:{userId} Uses TTL
- Thread Safety Explanation (VERY IMPORTANT) Redis ensures thread safety using atomic operations: • INCR is atomic → prevents race conditions Even with 200 concurrent requests, only 100 bot replies are allowed No possibility of overflow (e.g., 101 comments) This guarantees correctness under high concurrency.
- Phase 3: Notification Engine Redis Throttler • • Keys: If user received notification in last 15 min: → Store in Redis list Else: → Send notification immediately notif:{userId} pending:{userId}
- CRON Scheduler Runs every 5 minutes: Fetch pending notifications Aggregate them Log summary: Summarized Notification: X interactions Clear Redis list 3 Edge Cases Handled 1. Race Conditions • Solved using Redis atomic 2. Statelessness • • INCR No in-memory storage (no HashMap/static variables) All state managed via Redis 3. Data Integrity Flow: 1. Check Redis guardrails 2. If allowed → save in DB 3. If blocked → reject request
- Running the Project 1. Start Docker Services docker-compose up-d 2. Run Spring Boot App mvn spring-boot:run
- API Testing Use Postman collection provided.
- Test Cases : Create post Add human comment Add bot comment Like post Stress test bot limit (100 max)

- Project Structure src/main/java/com/example/guardrails ├── controller ├── service ├── repository ├── entity ├── config ├── scheduler
- Key Highlights : Redis-based concurrency control , Fully stateless architecture , Event-driven notification batching, Scalable microservice design
- Conclusion : This project demonstrates: Strong backend fundamentals Concurrency handling Distributed state management using Redis Clean API design with Spring Boot
  Author FAAZ KHAN